

v0.1 Text Agent 技术方案

☐ Minimax大模型

1.模块概述（技术需求分析）

Text Agent是v0.1版本中的核心信息抽取模块。给定一篇已选定的论文，将全文或者段落直接送入LLM，由LLM按照字段字典表1A和1B的定义完成结构化抽取，并输出标准JSON。本版本不引入RAG，全文上下文一次性传入。

2.技术目标

- JSON 格式合规、ID 格式合规、必填字段不为 null
- 核心溯源字段不为null
- 信息抽取准确率达到70%

3.输入规格

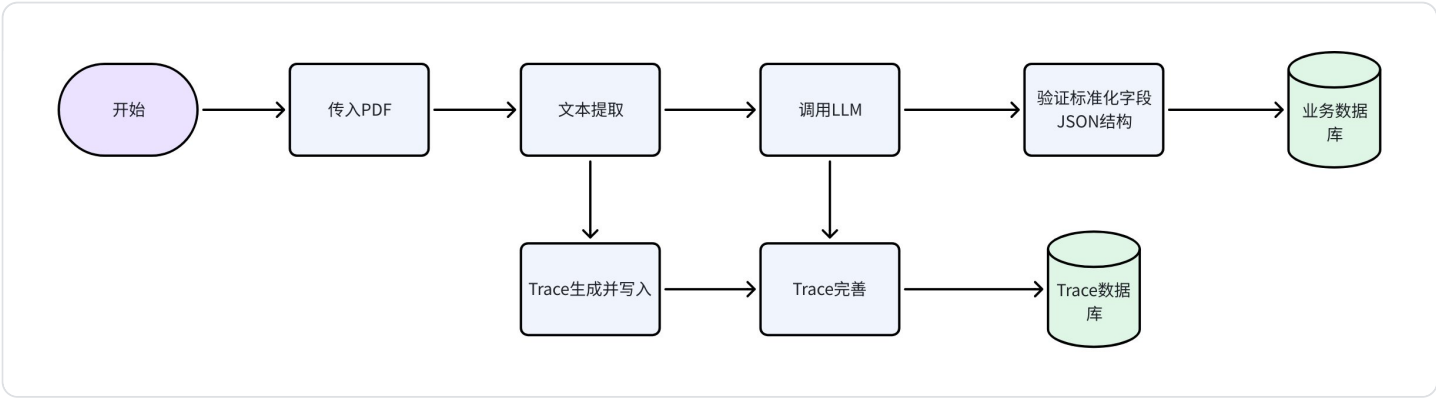
3.1输入规格

- v0.1由人工指定单篇论文，论文为PDF格式直接放在目录下的指定文件夹中
- 输入示例：./v0.1_paper/example.pdf

3.2输入限制

- v0.1部分不特别处理图片、表格、附录文件等其它内容
- v0.1部分不进行分块

4.方案设计



4.1方案概述

v0.1 采用线性单轮流程，不引入 Agent 编排框架。整体由三个模块串联组成：Text Agent 负责调用 LLM 完成信息抽取，数据库模块负责结构化结果的持久化存储，Trace 模块负责记录每次执行的完整过程。

系统使用两个独立的 SQLite 数据库文件，职责严格分离：

Code block

1	hap_v01.db	业务数据库	存储抽取结果 (paper_record + function_evidence)
2	hap_trace.db	Trace 数据库	存储执行过程 (extraction_runs + trace_steps)

两个数据库通过 extraction_run_id 关联，业务数据可随时追溯至对应的 Trace 记录

4.2数据流

☒ 使用OpenAI还是Anthropic还是国内其它的国产大模型

使用MiniMax最新的模型

阶段一：PDF 文本提取

使用 PyMuPDF（fitz）从 PDF 中提取纯文本。使用Claude可以通过Base64格式传入整篇PDF，但是使用Chatgpt就只能先把文本提取出来再传入。因为v0.1部分不涉及图片、表格等内容，所以这部分可以忽略。

阶段二：LLM 调用

将 PDF 文件与 Prompt 一并传入 LLM API，发起单次调用。系统记录调用时间戳、模型名称，等待响应。

阶段三：响应分叉

LLM 返回响应后，流程同时触发两条路径，两条路径使用同一份原始响应，互不依赖，写入顺序为先 Trace后业务：

Code block

```
1 LLM 响应
```

- 2 |——→ 业务路径：解析 JSON → 校验 Schema → 写入 hap_v01.db
- 3 |——→ Trace 路径：构造 Trace 记录 → 校验 Trace Schema → 写入 hap_trace.db

阶段四：Trace 写入

无论业务路径成功与否，Trace 均写入。Trace 记录包含本次调用的完整输入输出、模型信息、token 消耗、响应时间和 LLM 判断理由。业务路径的执行结果（成功/失败/错误类型）也一并记录至 Trace。

阶段五：业务数据写入

对 LLM 输出执行 JSON 解析和 Schema 校验。校验通过后，在同一事务内写入 paper_record（主表）和 function_evidence（子表）。校验失败时，记录原始响应，标记错误类型，跳过本次写入。

4.3 关键设计决策

Trace 无论成功失败都写入

Trace 的首要价值在于故障复盘。当业务写入失败时，Trace 里保存的原始响应和错误详情是定位问题的唯一依据。如果 Trace 只在成功时写入，失败场景反而没有记录，失去了 Trace 最重要的作用。

使用两个独立数据库文件

业务数据和 Trace 数据的生命周期不同。业务数据是长期积累的科研产出，需要严格维护；Trace 数据是过程日志，未来可能定期清理或归档。分开存储后，两者可以独立备份、独立迁移，互不影响。同时也避免 Trace 的高频写入对业务数据库造成锁竞争。

v0.1 只做单次 LLM 调用

v0.1 的目标是验证 LLM 能否从论文中抽取结构化数据，单次调用是最直接的验证方式。多轮对话、自动修复、重试逻辑会引入额外变量，干扰对 LLM 基础能力的判断。这些机制在能力验证通过后的 v0.2 再引入更合理

5. Prompt 设计

5.1 整体调用结构

v0.1 采用单次 API 调用完成全部抽取。论文全文通过 User Prompt 传入，System Prompt 包含任务定义和字段字典，两者共同构成完整请求。

`system` role 的定位 给模型设定"你是谁、你要做什么、你必须遵守什么规则"。模型会把 `system` 的内容视为高优先级的固定指令，不会把它当成对话内容来回应。

`user` role 的定位 给模型提供"这次具体要处理的内容"。模型会把 `user` 的内容视为当次任务的输入，结合 `system` 的指令来处理。

Code block

```
1 from openai import OpenAI
2
```

```

3  client = OpenAI() # 自动读取 OPENAI_API_KEY 环境变量
4
5  def call_llm(prompt_system: str, prompt_user: str) -> dict:
6      """
7      调用 OpenAI API 执行单次抽取。
8      返回完整 response 对象，由调用方提取内容和 token 信息。
9      """
10     response = client.chat.completions.create(
11         model="gpt-4o",
12         max_tokens=8000,
13         messages=[
14             {
15                 "role": "system",
16                 "content": prompt_system
17             },
18             {
19                 "role": "user",
20                 "content": prompt_user # 包含论文全文
21             }
22         ]
23     )
24     return response

```

5.2 System Prompt

System Prompt 为固定内容，每次调用不变。包含三部分：任务定义、输出规范、字段字典。

Code block

```

1  import json
2
3  def build_system_prompt(field_dict_path: str = 'field_dict.json') -> str:
4      """
5      构造 System Prompt。
6      读取 field_dict.json，序列化为 JSON 字符串后嵌入。
7      字段字典更新时只需修改 field_dict.json，此函数无需改动。
8      """
9      with open(field_dict_path, 'r', encoding='utf-8') as f:
10         field_dict = json.load(f)
11
12         field_dict_str = json.dumps(field_dict, ensure_ascii=False, indent=2)
13
14         return f"""
15 你是一个专业的生物医学文献结构化抽取助手。
16
17 任务：从用户提供的论文全文中，抽取与 HAp 相互作用肽段相关的结构化信息。

```

```
18  论文可能包含正文、图表描述和附表，请充分利用所有部分进行抽取。
19
20  输出格式：严格遵循 JSON Schema，不输出任何多余文字。
21  缺失字段：若论文中无对应信息，填写 null，不得猜测或捏造。
22  溯源要求：每个非 null 核心字段须在 text_to_xxx 字段中记录来源锚点。
23  判断理由：对 entity_type、sequence_status、evidence_overall_level、
24              trace_status 须在顶层 reasoning 对象中输出判断理由。
25
26  以下是字段字典，定义了每个字段的含义、类型和填写规则：
27  {field_dict_str}
28  """
```

5.3 User Prompt

User Prompt 由程序动态构造，每次调用随论文内容变化。论文全文通过 PyMuPDF 提取后直接拼入。

Code block

```
1  def build_user_prompt(extracted_text: str) -> str:
2      return f"""
3      请抽取以下论文中所有肽段 record。
4      每条 record 对应论文中一个对象级条目。
5      输出一个符合 System Prompt 中 JSON Schema 的完整 JSON 对象。
6
7      ===论文全文===
8      {extracted_text}
9      ===论文全文结束===
10     """
```

5.4 JSON Schema定义

以下 Schema 同时服务两个目的：一是约束 LLM 的输出结构（嵌入 System Prompt），二是作为后续 Schema 校验的依据

Code block

```
1  {
2      "paper_id": "string",                // 系统生成，LLM 不填
3      "reasoning": {                       // 仅写入 Trace，不入业务数据库
4          "entity_type_reason": "string",
5          "sequence_status_reason": "string",
6          "evidence_level_reason": "string",
7          "trace_status_reason": "string"
8      },
```

```

9      "records": [
10      {
11          "record_id":                "string",    // 系统生成, LLM 不填
12          "record_order_in_paper":    "integer",
13          "record_order_basis":        "sequence | text | table | figure |
curator",
14          "doi":                      "string",
15          "pmid":                     "string | null",
16          "title":                    "string",
17          "journal_title":             "string",
18          "publication_year":          "integer",
19          "first_author":              "string | null",
20          "entity_name_raw":           "string",
21          "entity_name_normalized":     "string",
22          "entity_type":               "peptide | protein | derivative |
chimera | polymer | mixture | unclear",
23          "sequence_raw":              "string | null",
24          "sequence_normalized":        "string | null",
25          "sequence_status":           "explicit | partial | not_reported |
backtrace_required",
26          "modification_raw":          "string | null",
27          "modification_normalized":    "string | null",
28          "modification_site_raw":      "string | null",
29          "source_origin":             "natural | synthetic | rational_design |
phage_dis | recombinant | computational | unclear",
30          "mw_reported":               "string | null",
31          "pl_reported":               "string | null",
32          "net_charge_reported":        "string | null",
33          "property_note":              "string | null",
34          "interaction_target":         "HAp | enamel | dentin | collagen | ACP
| mineral_surface | other | unclear",
35          "clinical_problem":           "string | null",
36          "summary_functions":          ["string"],
37          "effect_direction_summary":    "promote | inhibit | bind | stabilize |
mixed | unclear",
38          "mechanism_level_1":          "string | null",
39          "mechanism_level_2":          "string | null",
40          "evidence_overall_level":     "in_vitro | ex_vivo | animal | clinical
| in_silico | review_only | unclear",
41          "model_system_summary":       "string",
42          "assay_categories_summary":   ["string"],
43          "is_chimeric":                "yes | no | unclear",
44          "partner_module_raw":         "string | null",
45          "partner_module_normalized":   "string | null",
46          "partner_module_sequence_raw": "string | null",
47          "partner_module_length_aa":   "integer | null",
48          "partner_module_role":        "string | null",

```

```

49     "linker_raw":                "string | null",
50     "linker_normalized":         "string | null",
51     "linker_sequence_raw":       "string | null",
52     "linker_length_aa":          "integer | null",
53     "linker_type":               "flexible | rigid | spacer |
chemical_linker | peptide_linker | unclear | none | null",
54     "text_to_sequence":          "string | null",
55     "text_to_method":            "string | null",
56     "text_to_function_summary":   "string | null",
57     "text_to_evidence_summary":   "string | null",
58     "evidence_quote_short":       "string | null",
59     "trace_status":              "complete | partial | missing |
disputed",
60     "function_evidence": [
61         {
62             "function_evidence_id": "string",    // 系统生成, LLM 不填
63             "function_order_in_record": "integer",
64             "function_label":         "adsorption | mineralization |
remineralization | ACP_stabilization | nucleation | crystal_growth_modulation
| anti_demineralization | antimicrobial | other",
65             "effect_direction":       "promote | inhibit | bind | stabilize
| mixed | unclear",
66             "interaction_target":     "HAp | enamel | dentin | collagen |
ACP | mineral_surface | other | unclear | null",
67             "function_detail":        "string | null",
68             "evidence_level":         "in_vitro | ex_vivo | animal_in_vivo |
clinical | in_silico | review_only | unclear",
69             "model_system":           "string",
70             "assay_category_main":    "string",
71             "validation_method_main": "string | null",
72             "validation_method_secondary": "string | null",
73             "readout_main":           "string | null",
74             "result_summary":         "string | null",
75             "condition_pH":           "string | null",
76             "condition_solution_system": "string | null",
77             "condition_ionic_environment": "string | null",
78             "condition_challenge_type": "string | null",
79             "condition_time":         "string | null",
80             "condition_temperature":  "string | null",
81             "text_to_function":        "string",
82             "text_to_evidence":        "string",
83             "evidence_quote_short":    "string | null",
84             "trace_status":            "complete | partial | missing |
disputed"
85         }
86     ]
87 }

```

```
88     ]
89 }
```

6.其它执行部分

6.1 ID生成

paper_id、record_id、function_evidence_id 均由系统程序生成，覆盖 LLM 可能填写的任何值，确保格式合规率 100%

Code block

```
1  def generate_ids(paper_id: str, records: list) -> list:
2      """为每条 record 和 function_evidence 生成系统 ID，覆盖 LLM 填写的值。"""
3      for r_idx, record in enumerate(records, start=1):
4          record['paper_id'] = paper_id
5          record['record_id'] = f'{paper_id}-R{r_idx:02d}' # P000001-R01
6          for f_idx, fe in enumerate(record.get('function_evidence', []),
7              start=1):
8              fe['function_evidence_id'] = \
9                  f"{record['record_id']}-F{f_idx:02d}" # P000001-R01-F01
10         return records
```

6.2 完整执行框架

Code block

```
1  # text_agent.py
2  import time
3  from datetime import datetime, timezone
4  from extract import extract_text_from_pdf
5  from prompt_builder import build_system_prompt, build_user_prompt
6  from llm import call_llm
7  from parser import parse_response, validate_schema
8  from ids import generate_ids
9  from db_writer import insert_extraction_result
10 from trace_logger import TraceLogger
11
12 def run_text_agent(pdf_path: str, paper_id: str,
13                   business_conn, trace_conn):
14
15     # — 阶段 1: 初始化 Trace, INSERT extraction_runs —————
```



```

16     # run_id 在此生成, 后续所有操作都用这个 run_id 追踪
17     tracer = TraceLogger(trace_conn, paper_id, model_name='gpt-4o')
18     # 此时 extraction_runs 已写入:
19     #     run_id, paper_id, model_name, started_at(暂时为空),
20     #     run_status = "running"
21
22     # — 阶段 2: PDF 文本提取

```

```

23     extracted_text = extract_text_from_pdf(pdf_path)
24
25     # — 阶段 3: 构造 Prompt

```

```

26     prompt_system = build_system_prompt()
27     prompt_user   = build_user_prompt(extracted_text)
28
29     # — 阶段 4: 调用 LLM

```

```

30     # 调用前只记录时间戳, 不写 DB (排除 DB 写入时间的干扰)
31     t_call = time.time()
32     response = call_llm(prompt_system, prompt_user)
33     t_response = time.time()
34
35     # 收到响应后, 所有时间信息一次性计算
36     called_at      = datetime.fromtimestamp(t_call, timezone.utc).isoformat()
37     response_at    = datetime.fromtimestamp(t_response,
timezone.utc).isoformat()
38     response_ms    = int((t_response - t_call) * 1000)
39
40     raw_text      = response.choices[0].message.content
41     in_tokens     = response.usage.prompt_tokens
42     out_tokens    = response.usage.completion_tokens
43
44     # — 阶段 5: INSERT trace_steps (立刻落地, step_status 先占位) ——
45     step_id = tracer.insert_step(
46         prompt_system    = prompt_system,
47         prompt_user      = prompt_user,
48         raw_response     = raw_text,
49         input_tokens     = in_tokens,
50         output_tokens    = out_tokens,
51         called_at        = called_at,
52         response_at      = response_at,
53         response_time_ms = response_ms,
54         model_name       = 'gpt-4o',
55         step_status      = 'processing',    # 占位, 后续 UPDATE
56     )
57

```

58 # — 阶段 6: JSON 解析

```
59 parsed, reasoning = parse_response(raw_text)
60
61 # UPDATE trace_steps: 补充解析结果
62 tracer.update_step(step_id,
63     parsed_output = parsed,
64     llm_reasoning = reasoning,
65 )
66
67 if not parsed:
68     # UPDATE trace_steps: 标记解析失败
69     tracer.update_step(step_id,
70         step_status = 'parse_error',
71         error_detail = 'LLM output is not valid JSON',
72     )
73     tracer.finalize(run_status='failed',
74         error_message='JSON parse error')
75     return
```

76

77 # — 阶段 7: JSON Schema 校验

```
78 is_valid, error_detail = validate_schema(parsed)
79
80 if not is_valid:
81     # UPDATE trace_steps: 标记校验失败
82     tracer.update_step(step_id,
83         step_status = 'schema_error',
84         error_detail = error_detail,
85     )
86     tracer.finalize(run_status='failed',
87         error_message=f'Schema error: {error_detail}')
88     return
```

```
89
90 # UPDATE trace_steps: 校验通过
91 tracer.update_step(step_id, step_status='success')
```

```
92
93 # — 阶段 8: 生成 ID, 写入业务数据库 —————
94 records = generate_ids(paper_id, parsed['records'])
95 insert_extraction_result(business_conn, records, tracer.run_id)
```

96

97 # — 阶段 9: 完整性检查

```
98 check_status, check_detail = integrity_check(
99     business_conn,
100     paper_id,
101     expected_record_count = len(records)
```

```
102     )
103
104     # — 阶段 10: UPDATE extraction_runs, 写入最终状态 —————
105     tracer.finalize(
106         records_extracted      = len(records),
107         records_inserted       = len(records),
108         run_status              = 'success' if check_status == 'passed' else
    'failed',
109         integrity_check_status  = check_status,
110         integrity_check_detail  = check_detail,
111     )
112
```

7.错误处理

Text Agent 在执行过程中可能遇到以下五类错误。所有错误均写入 Trace，Trace 写入本身不受业务错误影响。

错误类型	触发场景	处理策略	是否重试	Trace记录
API调用超时	等待响应超过超时阈值	捕获 TimeoutError，记录至 Trace	最多重试 2 次，间隔 5s	step_status = timeout
API 返回错误状态码	HTTP 4xx / 5xx	记录状态码与响应体	4xx 不重试；5xx 重试 2 次	step_status = api_error
JSON 解析失败	LLM 输出非合法 JSON	记录原始响应，尝试自动修复，修复失败则跳过业务写入	否	step_status = parse_error
Schema 校验失败	enum 字段值超出允许范围，或必填字段为 null	记录具体不合规字段，尝试自动修复，修复失败则跳过业务写入	否	step_status = schema_error
PDF 文本提取失败	扫描版 PDF 或文件损坏	抛出异常，终止本次 run	否	run_status = failed

已知局限

局限	原因	计划版本
不支持扫描版 PDF	PyMuPDF 依赖 PDF 文字层	v0.2 引入 OCR 方案
不支持超长论文 (> 128k tokens)	GPT-4o 上下文窗口限制	v0.2 引入段落分块策略
不支持批量处理	v0.1 单篇验证，无任务队列	v0.3 引入批量处理流程
不支持图表信息抽取	PyMuPDF 纯文本提取不含图片内容	v0.2 引入 Figure/Table Agent
不做自动修复	v0.1 验证基础能力，避免引入额外变量	v0.2 视验证结果决定是否引入
不支持多模型	v0.1 单模型验证	v0.2 引入多模型适配层
token 消耗和质量基线未知	需要实际运行后才能得到数据	v0.1 跑完首批论文后填入质量基线